Practical:08	Implementation of graph searching using DFS and BFS.
__/__ /2026	

Objective:

- To implement Breadth First Search (BFS) and Depth First Search (DFS) for graph traversal using Python.
- To understand how a queue (BFS) and a stack (DFS) decide the order in which vertices are visited.
- To analyze the time and space complexity of both algorithms.

Program:

```
def bfs(graph, start): #
    BFS uses a queue
    visited = {start}
    queue = [start] order
    = [] while queue:
        node = queue.pop(0)
        order.append(node)
        # visit all unvisited neighbours for
        nb in graph[node]:
            if nb not in visited: visited.add(nb)
                queue.append(nb)
    return order

def dfs(graph, start): #
    DFS uses a stack
    visited = set()
    stack = [start]
    order = [] while
    stack:
```

```
        node = stack.pop() if node
        not in visited:
        visited.add(node)
        order.append(node)
            # push neighbours in reverse order
            for nb in reversed(graph[node]):
                if nb not in visited:
                    stack.append(nb)
    return order

# graph as adjacency list graph
= {
    'P': ['Q', 'R'],
    'Q': ['P', 'S', 'T'],
    'R': ['P', 'U'],
    'S': ['Q'],
    'T': ['Q', 'U', 'V'],
    'U': ['R', 'T'],
    'V': ['T']
}

print("BFS:", bfs(graph, 'P'))
print("DFS:", dfs(graph, 'P')) Output:
```

```
BFS: ['P', 'Q', 'R', 'S', 'T', 'U', 'V']
DFS: ['P', 'Q', 'S', 'T', 'U', 'R', 'V']
```

Analysis:

a) Problem Analysis

Graph traversal means visiting every vertex of a graph exactly once, starting from a given vertex. We store the graph as an adjacency list, where each vertex maps to the list of its neighbours.

The two traversals differ only in the data structure used to hold vertices that are waiting to be visited:

- **BFS** uses a queue (first in, first out) and visits vertices level by level.
- **DFS** uses a stack (last in, first out) and goes as deep as possible before backtracking. **b) Example Analysis**

Consider the graph with start vertex P:

Vertex	Neighbours
P	Q, R
Q	P, S, T
R	P, U
S	Q
T	Q, U, V
U	R, T
V	T

BFS Queue Trace:

Step	Processed	Queue after processing	Traversal
0	-	[P]	[]
1	P	[Q, R]	[P]
2	Q	[R, S, T]	[P, Q]
3	R	[S, T, U]	[P, Q, R]
4	S	[T, U]	[P, Q, R, S]
5	T	[U, V]	[P, Q, R, S, T]
6	U	[V]	[P, Q, R, S, T, U]
7	V	[]	[P, Q, R, S, T, U, V]

DFS Stack Trace (top of stack is at the right):

Step	Popped	Stack after processing	Traversal
0	-	[P]	[]
1	P	[R, Q]	[P]
2	Q	[R, T, S]	[P, Q]
3	S	[R, T]	[P, Q, S]
4	T	[R, V, U]	[P, Q, S, T]
5	U	[R, V, R]	[P, Q, S, T, U]

6	R	[R, V]	[P, Q, S, T, U, R]
7	V	[R]	[P, Q, S, T, U, R, V]
8	R	[]	already visited, skipped

BFS visits P, then its neighbours Q and R, then S, T and U, and finally V. DFS visits P, Q, S, then backtracks to Q and goes down to T, U, R and finally V. Vertex R is pushed on the stack twice (once from P and once from U); the second copy is skipped because R is already visited.

Therefore:

BFS traversal = P, Q, R, S, T, U, V

DFS traversal = P, Q, S, T, U, R, V

c) Algorithm Analysis

Step 1: Create the adjacency list of the graph.

Step 2: Create an empty visited set and the traversal list.

Step 3: BFS: put the start vertex in the queue and mark it visited. DFS: push the start vertex on the stack.

Step 4: Remove a vertex (front of queue for BFS, top of stack for DFS) and add it to the traversal.

Step 5: Add all its unvisited neighbours to the queue or stack. For DFS, add them in reverse order so they are explored in their original order.

Step 6: Repeat steps 4 and 5 until the queue or stack is empty.

d) Time Complexity

Each vertex is processed once, and every edge is checked a constant number of times while scanning the adjacency lists.

Therefore:

Time Complexity = O(V + E)

where:

- V = number of vertices
- E = number of edges

(Note: queue.pop(0) on a Python list takes O(n). Using collections.deque with popleft() keeps BFS at a true O(V + E).)

e) Space Complexity

The visited set, the traversal list, and the queue or stack can each hold at most V vertices.

Therefore:

Space Complexity = O(V)

Conclusion:

BFS and DFS were successfully implemented in Python. Starting from vertex P, BFS gives P, Q, R, S, T, U, V and DFS gives P, Q, S, T, U, R, V. Both algorithms visit every vertex once, with a time complexity of O(V + E) and a space complexity of O(V).